

Live video stat tracking — feasibility study

Date: 2026-10-09 · Status: for decision · App: Rotation Planner Client 4.8.0 · Proposed as the next major change

The question

Could the mobile app capture live video from the side of the court and, on its own, track and update the set and player stats the record screen takes by tap today: Serve In, Serve Out, Return In, Return Out, the set score, side-outs and substitutions? If yes, what would it take?

1. Short answer

Yes for part of it, with a coach still in the loop, and not from a single sideline phone for the hard calls. Here is the answer in five lines.

- **Rally boundaries, serves and dead balls** can be detected from one phone camera today. That alone removes most of the tapping, because the app already knows who is serving from the rotation.
- **Who won each rally**, and so the **score and side-outs**, can be got right most of the time. The best signals are not the ball at all: the first referee's point signal and a visible scoreboard. Ball tracking is the fallback, not the primary source.
- **In or out line calls** from one camera are only trustworthy for clear cases. Anything within about 30 cm of a line needs a confirmation tap. Hawk-Eye uses six to ten cameras in tennis, and more in volleyball, for a reason.
- **Substitutions** can be noticed, but reading which player came in needs jersey numbers at a resolution a sideline phone rarely gets. Subs stay a coach-confirmed event, with the sheet pre-filled.
- **The side of the court is the wrong first camera position** for line calls. An elevated end-line or corner view is what the commercial products insist on. The side view does have one real advantage: the referee's stand and the substitution zone are both in frame, one near the camera and the other across the court, which an end-line view cannot offer. The study recommends trialling both.

The cheapest first step costs almost nothing and de-risks everything else

Every tap the coach makes today is a time-stamped, labelled event. If the app records video while the coach taps as normal, every match produces a labelled training set for free. No model has to exist before that step starts. See Phase 0 in section 6.

The recommended path is **assisted capture**: the phone watches the match, proposes each event with a confidence, and the coach confirms with one tap or lets high-confidence events through. The app's existing undo and take-back sheet become the correction tool. Fully hands-off tracking of every stat from one phone is not achievable with current technology and should not be the target.

2. What each stat needs from the camera

The record screen's seven outputs are not equally hard. The table splits each into the visual signal it needs, how reliable that is from one phone, and whether a better signal exists.

Stat	What the camera has to see	From one phone	Better signal, and what the app already knows
Serve happened	Ball at rest behind the end line, then launched over the net. A whistle a second or two earlier.	Good	The app knows who is serving from the rotation and serve order, so no player identification is needed. Audio whistle detection is cheap and robust.
Serve In / Out	Whether the serve cleared the net and landed inside the far court, or the rally continued.	Mixed	"Rally continued" is easy (the ball comes back). "Landed out by 20 cm" is the hard case. Net faults are visible from the side. The referee's signal settles it when the serve fails.
Return In / Out	Which of our players received, and whether the ball stayed in play.	Mixed	"Stayed in play" is easy. Who passed is the problem: the app knows the six on court but not which one touched the ball. Player tracking plus receive-formation position gets a likely answer; a one-tap "who passed?" choice among the back row and libero closes the gap.
Rally winner (point)	Which side the ball died on, or the referee's arm.	Good with assist	The first referee extends an arm toward the side that won. A pose model on the referee's stand is far more reliable than tracking the ball to the floor. A visible scoreboard read by text recognition is a second check.
Set score	Nothing extra once rally winners are right.	Derived	The app already derives score from the rally log. Scoreboard reading can reconcile drift every few points.
Side-out	Nothing extra.	Derived	Already inferred from score and server (shipped in 4.3.0). Falls out for free.

Substitution	A player entering the substitution zone, a player leaving, and both jersey numbers.	Weak	Detecting <i>that</i> a sub happened is fine; the zone is on the scorer's sideline, near the camera if it stands on that side. Reading <i>which</i> players needs jersey text recognition, which fails on folds, occlusion and distance. Pre-fill the add-sub sheet and ask.
---------------------	---	-------------	--

Why the ball is hard

A volleyball is about 21 cm across. Float serves travel at 40 to 50 km/h at any level; jump serves reach 55 to 105 km/h in youth play and 95 to 110 km/h or more for elite men. At 30 frames per second a 70 km/h ball moves roughly 65 cm between frames, three ball widths, with motion blur under gym lighting. Phone cameras can run at 60 fps, but a browser app does not always get that, and fluorescent flicker adds frame-to-frame brightness changes. Modern small-object trackers (the TrackNet family, which looks at three consecutive frames and predicts a heat map) handle this in tennis and badminton, but they were trained on broadcast footage with a fixed high camera. Expect a fine-tuning effort, not a download.

Why in and out is hard from the side

Side view (what was asked)

Sees net height and net touches clearly. The first referee's stand is at the net post on the side opposite the scorer's table, and the substitution zone is on the scorer's side, so a side camera is close to one and about ten metres from the other, with both in frame. The near sideline is good. The far sideline is hidden behind players and foreshortened. The end lines are seen at a shallow angle, so depth along the court is poorly resolved. Mapping the floor (homography) from four tapped corners works, but the far half is imprecise.

Elevated end-line or corner view

Sees both sidelines and the far end line. The court is a clean trapezoid, so a four-corner tap gives a precise floor map. This is the position the commercial single-camera products require. It does not see the referee's arm as well, and the near end line is at the bottom edge of the frame.

A line call from one camera also needs the *moment* of ground contact, which is inferred from the ball's trajectory changing direction. With the ball partly hidden by a player at that moment, the estimate degrades fast. A reasonable target is: clear in or clear out right 90 to 95% of the time, within 30 cm of a line right 60 to 70% of the time, and those cases flagged for confirmation.

3. Where the app is today

The platform shapes the options more than the models do.

Platform	Plain HTML and JavaScript single-page app, no framework, no native wrapper (no Capacitor, Cordova, React Native). Installable to the home screen; no web manifest yet.
Build	Everything is inlined into one obfuscated HTML file in <code>dist/</code> and published to GitHub Pages. A service worker (cache v17) serves it offline.

Storage	<code>localStorage</code> only. There is no server, no account, no API. Stats leave as a <code>CIQS...</code> payload through the clipboard or the share sheet.
Camera, media, ML	None. The app has never asked for a permission. There is no video element, no Web Worker, no WebAssembly, no model.
Event model	Every tap goes through <code>tap()</code> in <code>src/session.js</code> , which updates the count, infers the rally letter, records the server, realigns the serve order and pushes an undoable history entry. Points go through <code>tapPoint()</code> . Score, side-outs and rotation are derived from that log, not stored.

Three consequences

1. **Automatic events must enter through the same functions as taps.** If the vision layer calls `tap()` and `tapPoint()` like a finger would, then undo, side-out inference, rotation tracking and the export payload keep working unchanged. The only new thing in the session model is a "source" and "confidence" on each history entry so the take-back sheet can show which events were automatic.
2. **Model weights cannot live inside the single inlined HTML.** A small detector is 5 to 25 MB. It has to be fetched as a separate file and cached by the service worker, which is a new class of asset for the build and deploy scripts.
3. **Any cloud processing is the first backend this product has ever had.** Upload, storage, accounts, cost and a privacy policy for video of players all arrive with it. That is the largest architectural decision in this study, larger than any model choice.

4. Architecture options

A. Everything on the phone, inside the web app possible, limited

Use the browser camera (`getUserMedia`) and run small models in the page with TensorFlow.js or ONNX Runtime Web, accelerated by WebGPU (Safari 26 and iOS 26 onward, current Chrome) and WebGL elsewhere. A nano-class detector at 640 px runs at an estimated 10 to 25 fps on a recent iPhone or flagship Android in the browser (no published benchmark; to be measured in Phase 1), which is enough for rally segmentation, player boxes and referee pose, and marginal for ball tracking.

- **For:** no backend, no video ever leaves the phone, fits the current product shape, offline.
- **Against:** the browser tab must stay in the foreground the whole match. Thermal throttling after 15 to 20 minutes is common. Memory limits in iOS Safari are tight. 60 fps capture is not guaranteed. Camera access from an installed home-screen app works on current iOS but has had regressions in the past.

B. Native wrapper with on-device models possible, new pipeline

Wrap the app in Capacitor and run the models through Core ML (iOS) or TensorFlow Lite (Android) with the camera at 60 fps and the neural accelerator. Two to four times the throughput of A, proper background and thermal management, app-store distribution.

- **For:** the best on-device accuracy; still no video leaves the phone.
- **Against:** an Xcode and Android Studio build, store review, signing, two native plugins to maintain. The single-camera limits on line calls do not change.

C. Record on the phone, process in the cloud most accurate, most new

The phone records and uploads video in 10 to 30 second chunks. A GPU service runs the heavy models (a mid-size detector, a TrackNet-style ball tracker, pose on the referee, text recognition on the scoreboard and jerseys) and returns events. Latency is near-live, not live.

- **For:** the models can be as big as they need to be; one codebase for the vision; improvements ship without an app update.
- **Against:** a backend, accounts, upload on gym Wi-Fi or mobile data (1 to 2 GB per hour at 720p), per-match GPU cost, and a privacy regime for video of players, many of them minors.

D. Assisted capture, hybrid recommended

The phone does the light work locally: whistle detection, rally start and end, the referee's arm, and a coarse ball track. It proposes each event with a confidence. The coach confirms with one tap, or lets events above a threshold through automatically. The heavy work, if any, runs after the set or in the cloud for a post-match pass that proposes corrections.

- **For:** the coach stays in control and the stats stay trustworthy from day one. Each step is shippable on its own. The existing take-back sheet is the correction tool. The cloud part can be deferred or skipped.
- **Against:** it is not hands-off, and it needs a new capture screen designed for one-handed confirmation while watching the court.

Criterion	A. In-browser	B. Native	C. Cloud	D. Hybrid
Rally and serve detection	Good	Good	Good	Good
Point winner	Fair to good	Good	Good	Good
Line calls near the line	Weak	Weak	Fair	Weak, but asked
Who passed	Weak	Fair	Fair	Asked
Substitution identity	Weak	Weak	Fair	Asked
New infrastructure	None	Native builds	Backend + GPU	Optional backend
Video leaves the phone	No	No	Yes	Only if opted in
Battery and heat over a match	Poor	Fair	Fair (upload)	Fair
Fit with the product today	Good	Poor	Poor	Good

5. What it would take

5.1 Data and labelling

- **Video of your own matches**, from both candidate camera positions, across at least two gyms and lighting conditions. Twenty to forty sets is a workable first corpus.
- **Labels from the app itself**. Serve, return, point and sub events with timestamps come from the coach's taps if the app records the clock alongside each history entry. That covers rally boundaries, winners and attribution with no manual labelling.
- **Manual labels for the ball and the court**: ball position in a few thousand frames (interpolation tools such as CVAT or Label Studio bring this to 1 to 2 seconds per frame), four court corners per fixed camera position, and referee arm signals. Public volleyball ball datasets exist on Roboflow and GitHub (the open-source VolleyVision work, for example) and can seed early experiments, but check licences first: VolleyVision's data and code are non-commercial and copyleft, so a shipped product will need its own labelled data.

5.2 Models

Component	Approach	Notes
Whistle detector	Audio classifier on the phone microphone (YAMNet-class embedding plus a tiny head).	Cheap, robust, runs anywhere.
Rally segmenter	Ball motion plus whistle plus player spread. Rally starts at the serve toss, ends when the ball stops or the whistle sounds.	First model to build.
Ball detector and tracker	Nano or small detector fine-tuned on volleyball, with a TrackNet-style three-frame tracker for the fast segments.	The most effort; the least needed early.
Court map	Four tapped corners at setup, homography to floor coordinates. Re-tap if the phone moves.	No learning needed.
Referee signal	Person detector locks on the referee's stand; a pose model reads the arm direction after each rally.	Best source of the winner.
Scoreboard reader	Text recognition on a user-drawn box around the scoreboard, sampled every few seconds.	Only where a scoreboard is visible.
Player tracker and jersey reader	Multi-object tracking of the six on court; jersey number recognition when visible.	Needed for "who passed" and subs. Last to build.

5.3 App work

- **Capture screen**: camera preview, court-corner calibration, a scoreboard box, a referee box, a big confirm and reject control usable one-handed, a confidence indicator.
- **Event source and confidence** on history entries; the take-back sheet shows automatic events distinctly and can reject them in one tap.
- **Clock on every history entry** and an optional video recording that keeps the match on the device. This is the Phase 0 change and is small.

- **Model loading:** separate weight files fetched and cached by the service worker; a loading state; a fallback to manual tapping when the device cannot run the models.
- **Battery, heat and storage** management: reduced resolution and frame rate when the device warms, a warning when storage is low, and a plain "video off, taps only" mode.
- **Permissions and privacy:** a first-run explanation of camera and microphone use, where video is stored, how long it is kept, and how to delete it.

5.4 Infrastructure (only for options C and D with cloud)

- Chunked upload, object storage, a GPU inference service (Replicate, Modal, RunPod or a cloud provider), a queue, and a results endpoint the app polls.
- Rough cost: on current GPU pricing, processing a one-hour match at reduced frame rate on a mid-range GPU is well under a few dollars. Storage of 1 to 2 GB per match dominates if video is retained.
- Accounts and authentication, which the app does not have, and a retention and deletion policy.

5.5 Privacy, consent and rules

- Filming youth sport needs consent from parents or guardians and usually permission from the venue and the league. Some competitions forbid sideline filming or restrict where a camera may stand.
- Keep video on the device by default, process locally, and make cloud upload an explicit opt-in per team. Never store video with the stats payload.
- The opposing team, officials and spectators are in frame too. Face blurring in any retained footage is advisable.

5.6 People and time

Rough ranges for one to two people with some machine-learning experience, part-time alongside the current app. They are estimates for sizing the decision, not commitments.

Phase	Outcome	Effort
0	Record video and clock-stamped taps; build the first corpus from real matches.	2 to 4 weeks
1	Post-match pass: rally segmentation, whistle and referee signal; proposes corrections to a recorded set.	2 to 4 months
2	Near-live assist on the phone: serve detection, point proposals with one-tap confirm, auto score and side-out.	3 to 5 months
3	Ball tracking for line calls, player tracking for attribution, substitution pre-fill; auto-accept above a confidence threshold.	6 months and open-ended

6. Phased roadmap

Phase 0 — Record alongside the taps [start here](#)

Add a clock to each history entry and an optional video recording on the record screen (the browser's MediaRecorder, saved to the device; Safari records MP4, Chrome records WebM, so the Phase 1 scripts must read both). The coach's workflow does not change. After a few match days there is a labelled

corpus from both camera positions, and a clear view of what the phone camera actually sees in your gyms. This phase also answers the camera-position question with evidence rather than argument.

Phase 1 — Post-match analysis

A desktop or cloud script processes recorded sets: whistle detection, rally boundaries and the referee's signal. It produces a proposed rally log, compared against the coach's taps. The output is accuracy numbers per signal, which decide whether Phase 2 is worth building. If the referee signal alone reaches 90% on point winners, the live product is viable.

Phase 2 — Near-live assist

The models from Phase 1 that proved accurate move onto the phone. The capture screen proposes serve and point events in real time; the coach confirms or rejects. Score and side-out update automatically through the existing code path. Tapping remains available at all times.

Phase 3 — Toward hands-off

Ball tracking for clear line calls, player tracking for receive attribution, substitution pre-fill, and auto-accept for events above a confidence threshold the coach sets. Line calls near the line and substitution identity remain confirmed events, by design.

7. Risks and open questions

- **Browser camera on iOS.** Camera access in installed web apps has worked since iOS 13.4 but has regressed in some releases. If it becomes unreliable, option B (native wrapper) becomes the on-device path.
- **Heat and battery.** Continuous camera and inference for a two-hour match will drain a phone and throttle it. A mount, a power bank and a reduced-frame-rate mode are part of the design, not optional extras.
- **Gym conditions.** Fluorescent flicker, low light, cluttered backgrounds, spectators walking through the frame and a phone knocked off its mount will all happen. The system must fail safe to manual tapping.
- **Referee variation.** Signals are standard, but stand position, lighting and partial occlusion vary by venue. Phase 1 measures this.
- **Accuracy expectations.** A coach who trusts automatic stats that are 85% right will stop checking. The confidence display and the take-back sheet exist to keep the human in the loop.
- **Buy rather than build.** Products such as Balltime (now part of Hudl) already produce volleyball stats from a single phone recording after the match. Licensing or integrating one may cover Phase 1 at lower cost, at the price of sending video to a third party and losing the live element.
- **Scope creep.** Once video exists, requests for clips, highlights and attack stats follow. The study scopes only the seven stats the record screen takes today.

8. Decisions needed

1. **Approve Phase 0.** It is small, changes nothing about how the coach works, and makes every later decision an evidence-based one.
2. **Camera position trial.** Film the same matches from the sideline and from an elevated end-line or corner, so Phase 1 can compare them.
3. **On-device only, or cloud allowed.** This sets the privacy posture and whether a backend is in scope. The recommendation is on-device first, cloud as a later opt-in.
4. **Target accuracy for auto-accept.** What confidence must an event reach before it goes in without a tap? This is a product decision the coach should own.

What this study does not promise

Fully automatic in-and-out calls near the line, automatic identification of which player passed, and automatic substitution identity are not achievable from one phone with acceptable reliability today. The roadmap keeps those as confirmed events and treats any later improvement as a bonus.